

S O U T E N A N C E T I T R E P R O F E S S I O N N E L

DevOps Monitoring Platform

Plateforme d'observabilité conteneurisée pour la supervision distribuée

Yassine TAIFI BERNOUSSI

Candidat au titre Administrateur Systèmes DevOps (TP-01893)

Mai 2026

Cinq parties, vingt minutes

1

Contexte et problématique

Pourquoi l'observabilité ? Quel besoin ?

2

Architecture et stack technique

Trois piliers, douze services orchestrés

3

Réalisations par bloc REAC

Bloc 1 • Bloc 2 • Bloc 3

4

Démonstration live

Probes, dashboards, alerte Discord en direct

5

Bilan, perspectives et questions

Couverture compétences, suite en production

Sans observabilité, le MTTR explose

LE CONSTAT

Applications cloud distribuées — microservices, conteneurs, dépendances externes multiples.

Sans outil de supervision : **détection tardive** des incidents, origine non identifiée, logs éparpillés, alertes manquées.

Conséquence : **le MTTR** (Mean Time To Resolve) passe de quelques minutes à plusieurs heures.

99,8 %

Probe Uptime SLA

cible métier de la plateforme

< 1 min

Détection incident

cible métier de la plateforme

30 s

Scrape Prometheus

cible métier de la plateforme

Les trois piliers de l'observabilité moderne

Une plateforme conteneurisée qui unifie métriques, logs et traces avec corrélation native.



MÉTRIQUES

Prometheus + Grafana

Scrape 15s du backend FastAPI, node-exporter, cAdvisor.
Rétention 15 jours, dashboards provisionnés.

6 KPIs métier custom



LOGS

Elasticsearch + Logstash + Kibana

Logs JSON structurés émis vers Logstash via TCP.
Handler tolérant aux pannes, indexation par service/jour.

trace_id injecté dans chaque log



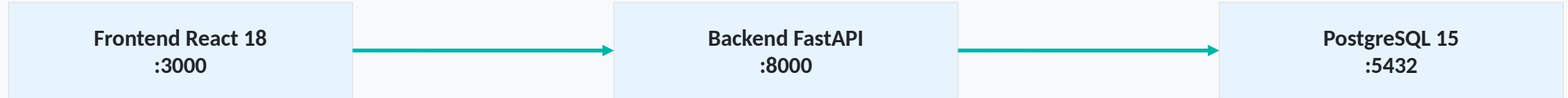
TRACES

Jaeger + OpenTelemetry

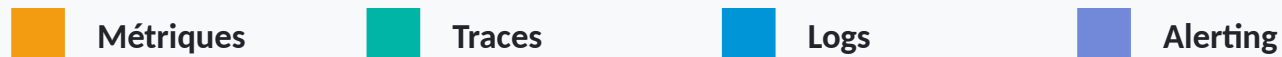
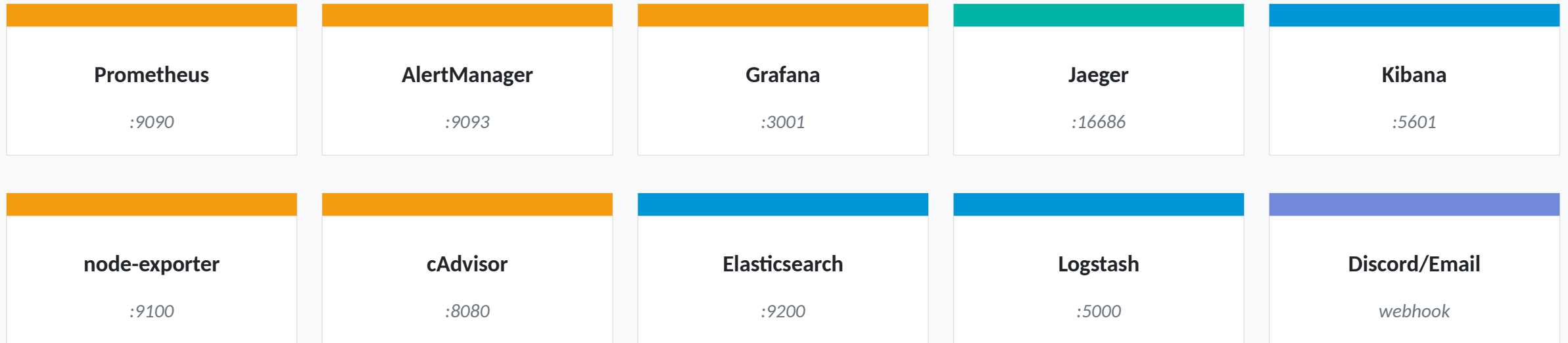
OTLP gRPC port 4317. Instrumentation automatique
FastAPI, Requests, Logging par OTel SDK.

Corrélation logs ↔ traces

12 services orchestrés en Docker Compose



OBSERVABILITÉ



Open-source de A à Z, 100 % conteneurisé

Couche	Technologies
Application	FastAPI 0.110 (Python 3.11), React 18 + Vite, JWT bcrypt, APScheduler
Stockage	PostgreSQL 15-alpine (volume nommé), Elasticsearch 8.13
Métriques	Prometheus 2.51 + AlertManager v0.27 + node-exporter v1.7 + cAdvisor
Dashboards	Grafana 10.4 — datasources et dashboards provisionnés (YAML/JSON)
Tracing	Jaeger 1.55 all-in-one + OpenTelemetry SDK Python 1.24 (OTLP gRPC)
Logs	ELK Stack 8.13 (Elasticsearch + Logstash + Kibana)
Infrastructure	Terraform 1.6 (provider AWS 5.x), Docker Compose v2
CI/CD	GitHub Actions (Ruff + Pylint + pytest + Trivy + Terraform validate)
Notifications	Discord webhook + SMTP Gmail (App Password)

Provisionnement AWS par Infrastructure as Code

CE QUE J'AI FAIT

311 lignes de Terraform (vs 6 lignes factices au départ).

- VPC dédié + IGW + Route Table + Subnet public
- Security Group restrictif (8 ports)
- EC2 t3.micro Amazon Linux 2023, **AMI dynamique** via data "aws_ami"
- EBS gp3 30 Go **chiffré au repos** (encrypted=true)
- **ssh_cidr secure-by-default 127.0.0.1/32** avec validation refusant 0.0.0.0/0
- user_data Bash : Docker + Compose + clone Git + docker compose up -d

COMPÉTENCES VALIDÉES

B1-CP2

Mettre en œuvre l'IaC

311 lignes HCL versionnées

B1-CP3

Sécuriser l'infrastructure

EBS chiffré, SG restrictif, ssh_cidr validation

B1-CP4

Mettre en production

user_data EC2 → stack auto-déployée

B1-CP1

Scripts d'automatisation

Bash user_data + clone Git

Conteneurisation et orchestration Docker Compose

12

services orchestrés

5

volumes nommés

multi-stage

Dockerfile non-root

11/11

tests pytest passés

SÉCURITÉ — Dockerfile multi-stage non-root

- ✓ Image runtime sans gcc/build-essential → **surface d'attaque réduite** (taille ÷ 3)
- ✓ Utilisateur **appuser non-root** → limite l'impact d'une RCE
- ✓ HEALTHCHECK natif → l'orchestrateur détecte et redémarre un conteneur défaillant
- ✓ Scan Trivy automatique en CI (CRITICAL/HIGH bloquant)
- ✓ DATABASE_URL via \${VAR:?} → **refus de démarrer si secret absent**

VALIDE

B1-CP3

Sécurité

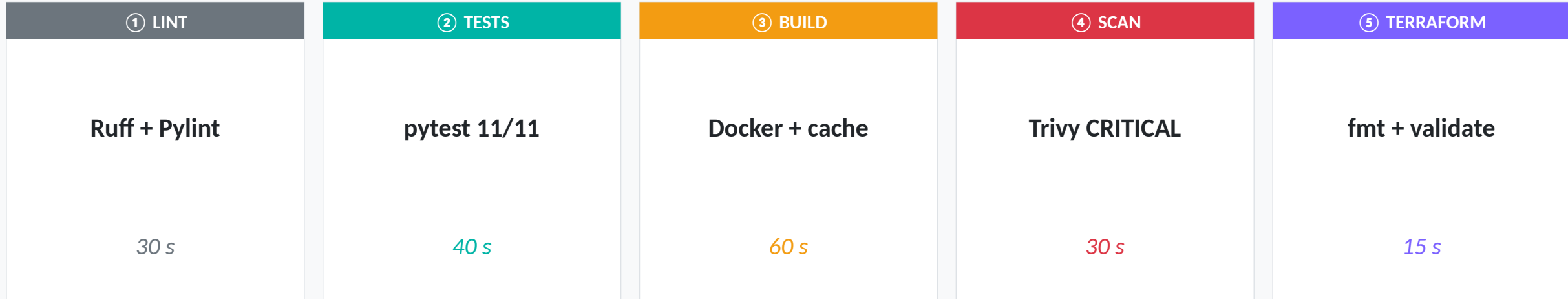
B1-CP4

Mise en production

B2-CP3

Conteneurisation

Pipeline CI/CD GitHub Actions à 5 jobs



Tous les jobs verts à chaque push sur main. Détection des régressions sous 5 minutes.

DIFFICULTÉS RÉSOLUES EN CI

httpx ≥ 0.28 cassait TestClient Starlette → pin httpx==0.27.2
passlib 1.7.4 incompatible avec **bcrypt 4.x** → pin bcrypt==3.2.2
Terraform validate sans credentials AWS → step shell tolérant (exit 0)

B2-CP4

Intégration et
livraison continues

Pipeline réel, scan sécurité, artefacts versionnés.

Stockage adapté : PostgreSQL + 5 volumes Docker

Service	Stockage retenu	Justification
Backend	PostgreSQL 15 + volume db_data	ACID, durée longue, JSON natif
Elasticsearch	Volume es_data	Index volumineux, durée longue
Prometheus	Volume prometheus_data	Time-series, rétention 15 jours
Grafana	Volume grafana_data	Dashboards et paramètres
AlertManager	Volume alertmanager_data	État des alertes
Postgres de test	tmpfs (RAM)	Volatile, conforme PSSI

VOLUME DOCKER vs bind mount

- ✓ Portable Windows ⇔ Linux
- ✓ Cycle de vie géré par Docker
- ✓ Pas de chemin host à maintenir
- ✓ Sauvegardes via docker run

B2-CP2

Concevoir et mettre en œuvre une solution de stockage adaptée

+ retrait du monitoring.db SQLite + script git filter-repo pour nettoyer l'historique.

Stack d'observabilité — trois piliers corrélés

MÉTRIQUES

Prometheus + Grafana

- Scrape 15 s
- Rétention 15 j
- 5 jobs configurés
- 11 alertes en 3 groupes

LOGS

ELK 8.13

- JSON structuré (stdout + TCP)
- Logstash filters
- Index par service + jour
- Discover Kibana

TRACES

Jaeger + OpenTelemetry

- OTLP gRPC :4317
- Auto-instrumentation FastAPI
- trace_id injecté dans logs
- Visualisation timeline

Corrélation logs ↔ traces : trace_id OpenTelemetry injecté dans chaque enregistrement de log JSON

KPIs métier, alerting Discord, capacity planning

« Le jury n'évalue pas "vous avez Prometheus", il évalue "vous racontez votre métier avec les chiffres". »

99,8 %

Uptime SLA

6

KPIs métier custom

5

alertes business

30 s

Scheduler APScheduler

MÉTRIQUES CUSTOM EXPOSÉES

sites_monitored_total sites surveillés
sites_up_count / down_count statut temps réel
monitoring_uptime_ratio SLA gauge
site_status{site} 1=UP, 0.5=DEGRADED, 0=DOWN
site_check_total{site,status} counter
site_check_latency_seconds histogram p50/p95/p99

ALERTES MÉTIER + Discord

- **ProbeUptimeSLABreach**
- **SiteDownPersistent**
- **SiteHighLatency (par site)**
- **ProbeFailureRateHigh**
- **NoMonitoringDataReceived**
 - Receiver Discord webhook + email SMTP de secours
 - Chaque alerte annotée d'un champ « decision »

Quelle partie du projet valide quel bloc REAC ?

Bloc	Compétence	Partie du projet qui la valide
B1	CP1 — Scripts d'automatisation	user_data Bash + scripts/clean-git-history.sh + CI GitHub Actions
B1	CP2 — Mettre en œuvre l'IaC	terraform/main.tf — 311 lignes (VPC + EC2 + SG + AMI dynamique)
B1	CP3 — Sécuriser l'infrastructure	Multi-stage non-root + EBS chiffré + SG restrictif + ssh_cidr validation + Trivy CI
B1	CP4 — Mettre en production	Docker Compose 12 services + user_data EC2 auto-deploy
B2	CP1 — Env de test isolé	docker-compose.test.yml + tmpfs + conftest pytest
B2	CP2 — Solution de stockage	PostgreSQL 15 + 5 volumes Docker nommés + retrait SQLite
B2	CP3 — Conteneurisation	Dockerfile multi-stage + healthcheck + DATABASE_URL via .env
B2	CP4 — CI/CD	GitHub Actions 5 jobs (lint + tests + build + Trivy + Terraform)
B3	CP1 — KPIs	6 métriques métier + 5 alertes business + scheduler APScheduler
B3	CP2 — Supervision	3 piliers (Prometheus + Jaeger + ELK) + AlertManager → Discord + capacity planning

Ce que le projet livre concrètement

12

services orchestrés

11

*alertes définies
(4 infra + 2 app + 5 métier)*

311

*lignes Terraform
fonctionnel (vs 6 au départ)*

11/11

tests pytest passés

6

KPIs métier custom

~74 %

*couverture pytest
sur composants critiques*

5

jobs CI/CD GitHub Actions

0

*vulnérabilités CRITICAL
remontées par Trivy*

+ 7 documents Markdown (CDC, architecture, business-metrics, RUNBOOK, README, terraform/README, scripts/README)

Cinq pièges techniques identifiés et résolus



AMI Terraform hardcodée ami-12345678 inexistante

Problème détecté



Migration vers data "aws_ami" avec filtres

Solution appliquée et tracée en commit Git



passlib 1.7.4 incompatible avec bcrypt 4.x

Problème détecté



Pin bcrypt==3.2.2 dans requirements.txt

Solution appliquée et tracée en commit Git



httpx ≥ 0.28 casse TestClient Starlette

Problème détecté



Pin httpx==0.27.2

Solution appliquée et tracée en commit Git



node-exporter plantait sur Docker Desktop Windows

Problème détecté



Mise sous profile Docker Compose "linux"

Solution appliquée et tracée en commit Git



monitoring.db SQLite versionné dans Git

Problème détecté



git rm + .gitignore + script git filter-repo

Solution appliquée et tracée en commit Git

PARTIE 4

Démonstration live

Stack lancée en local · Grafana · Jaeger · Kibana · Alerte Discord déclenchée en direct

01

docker compose ps

Tous les services UP / healthy

02

Grafana — Business Metrics

16 panels live, uptime SLA, latence par site

03

Jaeger UI

Traces du backend (corrélation logs)

04

docker compose stop logstash

Alerte InstanceDown reçue sur Discord → 70 s

Ce qui marche aujourd'hui, ce qui évoluerait en production

ACQUIS DU PROJET

- ✓ **10/12 compétences REAC couvertes** (les 2 restantes = oral)
- ✓ Stack 100 % open-source, reproductible
- ✓ Tout est versionné Git (code + config + dashboards)
- ✓ Pipeline CI/CD **tous jobs verts**
- ✓ Sécurité multi-niveaux : non-root, EBS chiffré, scan Trivy, secrets via .env
- ✓ Approche métier (BC03-CP2) : forecast et décisions, pas juste « CPU 45 % »

PERSPECTIVES PRODUCTION

- Backend Terraform distant (S3 + DynamoDB lock)
- Migration Kubernetes + Helm (charts officiels Prometheus, Grafana, ELK)
- Application Load Balancer + ACM (HTTPS)
- Secrets via AWS SSM Parameter Store ou Vault
- HPA basé sur métriques Prometheus métier (uptime_ratio)
- Service mesh (Istio) pour mTLS interne



Questions

Je reste à votre disposition pour échanger sur les choix techniques, les difficultés résolues et les perspectives.

Yassine TAIFI BERNOUSSI · yassinetaifi905@gmail.com
github.com/YTA048/Devops-monitoring

Merci pour votre attention.